Mission 2: The Lesion Engine

The story

"08:10. An ambulance crew calls the hospital: the patient cannot speak and the right arm is weak. The doctor on duty suspects a clot. Before the patient arrives, the doctor wants to know: if this pipe is blocked, what else stops working? Your second mission is the engine that answers that question."

A *lesion* is the doctors' word for damaged brain tissue. That is all the word means here.

What you need to know

Mission 1 gave the idea. Your program must follow these four exact rules:

1. The heart always has blood. It can never be blocked.
2. A blocked pipe has no blood.
3. Any other pipe has blood if **at least one** of the pipes it comes from has blood.
4. An area works if **at least one** of the pipes that feed it has blood.

The base challenge

Your goal: when a doctor names a blocked pipe, Synapse says what stops working. When a doctor names an area, Synapse says which pipes put it at risk.

Goal 1: Block a pipe

`lesion block <pipe>` blocks one pipe and prints:

- the blocked pipe and its size,
- every pipe left without blood,
- every area that stops working, with what it did,
- the expected symptoms: the symptoms of those areas, each listed once.

List pipes and areas in the order they appear in the atlas file.

Example CLI interaction (you may design yours differently):

```
synapse> lesion block left-middle
Blocked pipe: left-middle (big)
Pipes without blood (3): left-middle, left-middle-top, left-middle-bottom
Areas that stop working (3):
  speaking         Turns thoughts into words
  right-arm-face   Moves the right arm and face
  understanding    Understands what other people say
Expected symptoms (4): cannot-speak, right-arm-weak, right-face-droop, cannot-understand

synapse> lesion block back-main
Blocked pipe: back-main (big)
Pipes without blood (3): back-main, left-back, right-back
Areas that stop working (4):
  right-vision     Sees everything on the right side
  left-vision      Sees everything on the left side
  memory           Stores new memories
  awake            Keeps the patient awake and alert
Expected symptoms (4): blind-on-right, blind-on-left, forgets-new-things, drowsy

synapse> lesion block back-left-neck
Blocked pipe: back-left-neck (small)
Pipes without blood (1): back-left-neck
Areas that stop working (0): none
```

Make sure you understand the last example before you write any code.

Goal 2: Find the risky pipes for an area

A doctor sees a patient who cannot speak and asks: which pipe could be blocked? `lesion risk <area>` lists every single pipe that, if it alone were blocked, would stop that area.

The two commands must always agree with each other.

```
synapse> lesion risk speaking
3 pipes stop speaking if blocked:
  left-neck         big
  left-middle       big
  left-middle-top   small

synapse> lesion risk memory
1 pipe stops memory if blocked:
  back-main         big

synapse> lesion risk balance
No single blocked pipe can stop balance.
```

Error handling

Your program must handle and clearly report errors such as:

- "ERROR: Missing pipe. Usage: lesion block <pipe>"
- "ERROR: No pipe called "left-midle"."
- "ERROR: The heart cannot be blocked."
- "ERROR: "speaking" is an area, not a pipe. Only pipes can be blocked."
- "ERROR: "left-middle" is a pipe, not an area. Usage: lesion risk <area>"



- "ERROR: No atlas is loaded. Use: atlas load <path>"

Names in commands ignore upper and lower case: `LEFT-MIDDLE` works the same as `left-middle`.

The bonus challenge

For extra points.

Bonus 1: Several clots at once

`lesion block` accepts a list of pipes separated by commas.

```
synapse> lesion block back-left-neck, back-right-neck
Blocked pipes: back-left-neck, back-right-neck
Pipes without blood (5): back-left-neck, back-right-neck, back-main, left-back, right-back
Areas that stop working (5): right-vision, left-vision, memory, awake, balance
Expected symptoms (5): blind-on-right, blind-on-left, forgets-new-things, drowsy, falls-over
```

Bonus 2: Bridges

In a real brain, a few short pipes connect two pipes so blood can take a detour. We give you `data/atlas-bridges.txt`: the same atlas plus a new kind of record.

```
BRIDGE | left-front | right-front
BRIDGE | left-neck  | left-back
BRIDGE | right-neck | right-back
```

Blood can cross a bridge in either direction. Rule 3 becomes: a pipe has blood if at least one pipe it comes from **or is bridged to** has blood.

```
synapse> atlas load data/atlas-bridges.txt
SUCCESS: Atlas loaded from data/atlas-bridges.txt: 16 pipes, 12 areas, 3 bridges.

synapse> lesion block back-main
Blocked pipe: back-main (big)
Pipes without blood (1): back-main
Areas that stop working (1):
  awake            Keeps the patient awake and alert
Expected symptoms (1): drowsy
Saved by a bridge: left-back (from left-neck), right-back (from right-neck)
```

Bonus 3: Show it on the tree

`lesion tree <pipe>` prints the tree from Mission 1, Bonus 1, and marks what the clot affects.

```
synapse> lesion tree left-middle
heart
  left-neck
    left-middle              BLOCKED
      left-middle-top        no blood
        [speaking]           stops working
        [right-arm-face]     stops working
      left-middle-bottom     no blood
        [understanding]      stops working
    left-front
      [right-leg]
  ...
```




Hints

- Draw the atlas on paper and block a few pipes by hand before you write any code.
- Some pipes and areas get blood from more than one pipe. Read the four rules again and check that your program follows them there too.
- For Bonus 2: blood can now flow in a circle. Make sure your program always finishes.

What the judges will test

- Every pipe in `atlas.txt` blocked one at a time, compared with the judges' answer sheet.
- The same on the hidden `atlas-v2.txt`, with no code changes.
- `lesion risk` for every area, including areas no single pipe can stop.
- That the blood rules are written once in your code, not once per command.

What this mission saves

Nothing goes into your database yet. A later mission relies on your lesion engine, so build it as one piece of code that other commands can call.